Azure Hub-and-Spoke Topologies - Terraform & GitHub Actions

Terraform CI passing | Deploy — Layer 1 (Manual Peering) passing | Deploy — Layer 2 (AVNM) passing | Deploy — Layer 3 (vWAN) passing

Azure Cloud | Terraform >=1.5 | Auth OIDC Federated | last commit yesterday | License MIT

A portfolio project implementing three **production-grade Hub-and-Spoke network architectures** on Microsoft Azure, evolving from manual VNet Peering through AVNM policy-driven automation to a fully managed Virtual WAN - each layer provisioned via **Terraform** and validated through **GitHub Actions CI/CD** with **OIDC federated authentication**.

> **Note:** This project is intended for **learning, portfolio, and architectural validation purposes**. It is **not** designed for production deployments.

> **Suggested reading order:** Start from Layer 1 RUNBOOK and progress sequentially - each layer builds on design decisions and lessons learned from the previous one.

Table of Contents

- Azure Hub-and-Spoke Topologies - Terraform & GitHub Actions
  - Table of Contents
  - Architecture Overview
  - Skills Demonstrated
  - Layer Comparison
  - Cost Analysis
    - Summary of Cost Drivers
  - Phase 0: Project Foundation
    - Zero-Trust Authentication (OIDC)
    - Remote Terraform State
    - Continuous Integration (CI)
  - Layer Details
    - Layer 1 - Manual VNet Peering
    - Layer 2 - Azure Virtual Network Manager (AVNM)
    - Layer 3 - Azure Virtual WAN (vWAN)
  - Quick Start
    - Prerequisites
    - Deploy a Layer
  - CI/CD Workflows
  - Known Limitations
  - Repository Structure
  - Tech Stack
  - License

Architecture Overview

This project implements the same logical security objective across three progressive layers: all spoke-to-spoke traffic and internet egress is forced through a central **Azure Firewall** acting as the NVA. The enforcement mechanism, topology model, and operational complexity evolve at each layer.

> Each layer topology diagram is generated via Azure Network Watcher and documented in the respective `RUNBOOK.md`.

Layer	Technology	Enforcement Mechanism	Best For
Layer 1	Manual VNet Peering	Explicit UDRs per spoke subnet	Small footprints, full routing control
Layer 2	Azure Virtual Network Manager	Tag-based dynamic policy membership	Scaling spoke onboarding without peering loops
Layer 3	Azure Virtual WAN	Routing Intent on Secured Virtual Hub	Global transit, multi-region, managed NVA

Skills Demonstrated

Area	Detail
Azure Networking	VNet Peering, UDR engineering, AVNM dynamic policy membership, vWAN Routing Intent, Azure Firewall Standard, Azure Bastion (Basic/Standard SKU)
Infrastructure as Code	Terraform modules, remote state backends, `azurerm` provider
CI/CD & Automation	GitHub Actions matrix pipeline, parallel multi-layer validation, OIDC federated auth (zero static secrets)
Zero-Trust Design	No long-lived credentials, Azure Firewall as NVA, forced traffic inspection on all East-West and egress flows
Observability	KQL queries on Log Analytics (`AzureFirewallNetworkRule`), Network Watcher (Next Hop, Connection Troubleshoot, Effective Routes)

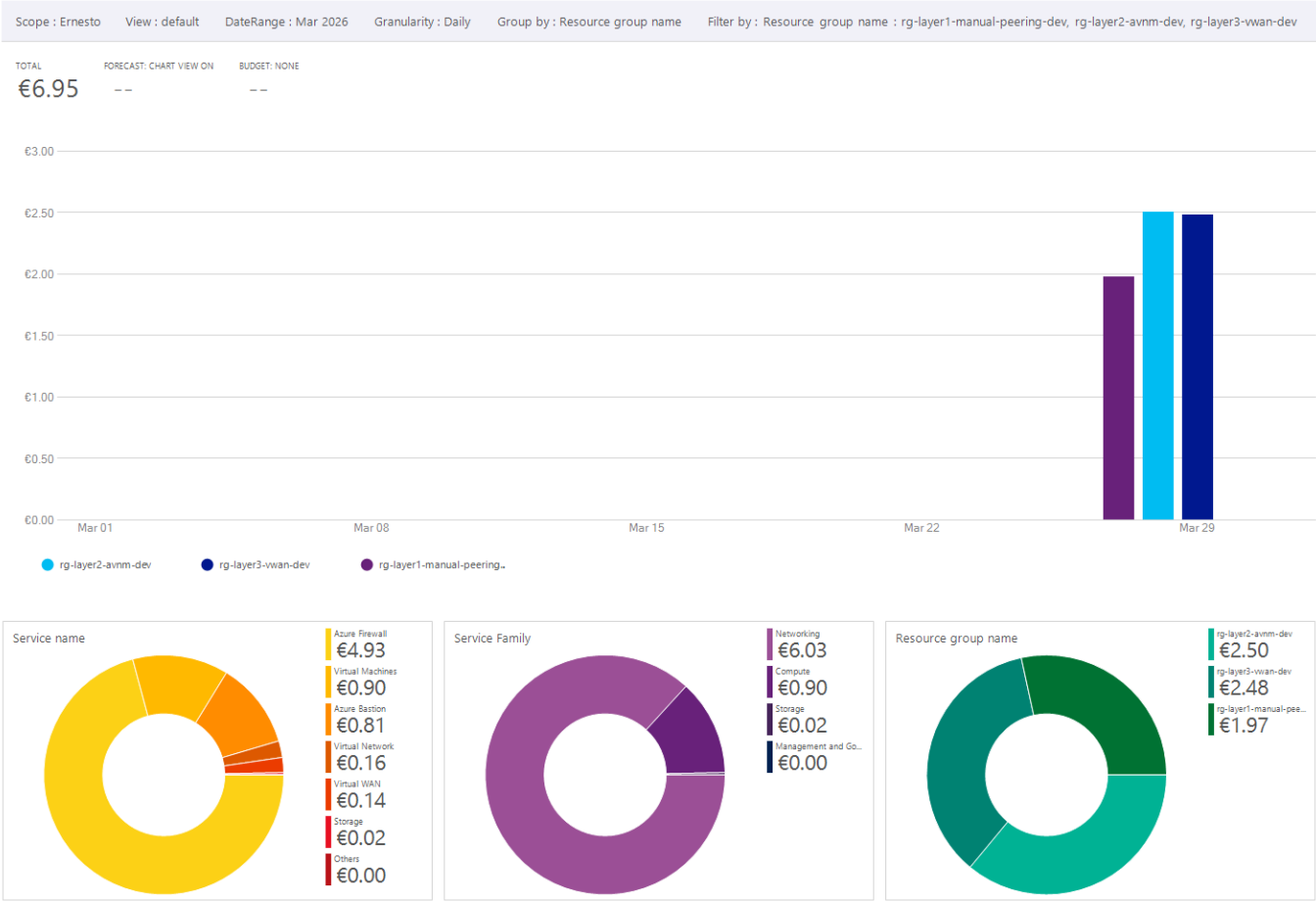
Layer Comparison

Feature	Layer 1	Layer 2	Layer 3
Spoke onboarding	Manual peering per spoke	Tag `avnm-group` on VNet	Virtual Hub Connection
UDR required	Yes - per spoke subnet	Yes - per spoke subnet	No - Routing Intent handles it
Dynamic membership	No	Yes - Azure Policy-based	Yes - native Hub attachment

Feature	Layer 1	Layer 2	Layer 3
Provisioning time	~3 min	~15-20 min	~30-45 min
Multi-region native	No	No	Yes
Custom routing control	Full	Full	Partial (Routing Intent is blanket)

Cost Analysis

A breakdown of the infrastructure costs incurred during the validation of the three architectural layers. Total spend was localized to short-lived laboratory deployments (resources were applied, tested, and subsequently destroyed).



Summary of Cost Drivers

- **Networking Dominates Spend:** Networking services account for over **86%** of the overall cost (€6.03 out of €6.95). In comparison, compute resources (test VMs) represent a marginal portion (€0.90).
- **Azure Firewall as the Principal Weight:** Responsible for nearly **71%** of the total bill (€4.93), the Azure Firewall Standard SKU is the most expensive resource. This emphasizes that strict Hub-and-Spoke and Zero-Trust topologies offload complexity and security to NVA appliances at a significant premium.
- **Azure Bastion:** Using Azure Bastion accounted for €0.81. The requirement to use Bastion **Standard SKU** (instead of Basic) for reliable IP-based connections across Layer 3 segments introduced a notable secondary fixed cost.

- **Cost Scaling and Uptime:**

Layer	Cost	Uptime	Avg €/hr
Layer 1 - Manual Peering	€1.97	~1h 50m	~€1.07/hr
Layer 2 - AVNM	€2.50	~2h 28m	~€1.01/hr
Layer 3 - Virtual WAN	€2.48	~2h 15m	~€1.10/hr
Total	**€6.95**		

Phase 0: Project Foundation

Before deploying any network layer, two foundation pieces were established once via the Azure CLI.

Zero-Trust Authentication (OIDC)

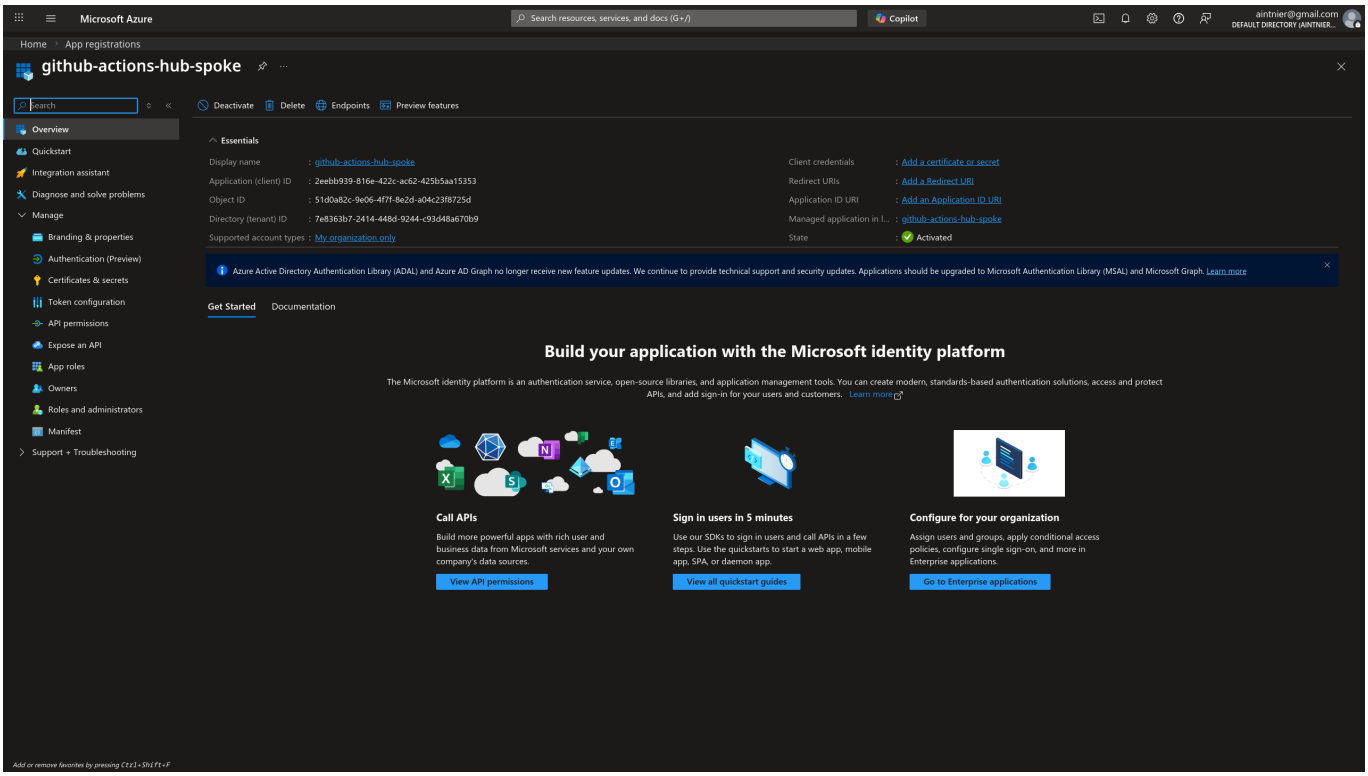
This project does **not** use long-lived client secrets. GitHub Actions authenticates to Azure exclusively through **OpenID Connect (OIDC) Federated Identity**.

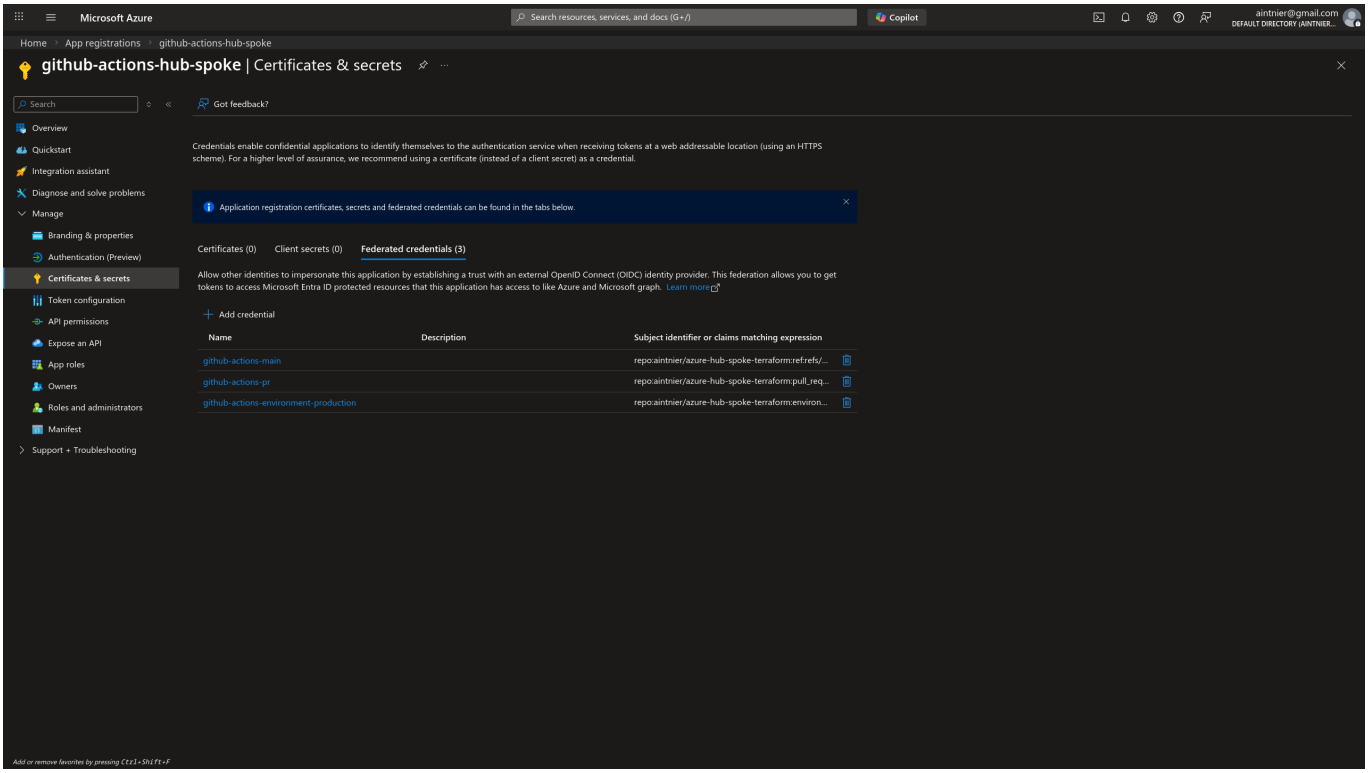
Federated Credential	Subject
`github-actions-main`	`repo:aintnier/azure-hub-spoke-terraform:ref:refs/heads/main`
`github-actions-pr`	`repo:aintnier/azure-hub-spoke-terraform:pull_request`
`github-actions-environment-production`	`repo:aintnier/azure-hub-spoke-terraform:environment:production`

▶ Azure CLI setup commands

```
az ad app create --display-name "github-actions-hub-spoke"
az ad sp create --id <APP_ID>
az role assignment create --assignee <SP_ID> --role Contributor --scope
/subscriptions/<SUB_ID>

az ad app federated-credential create \
  --id <APP_ID> \
  --parameters '{"name":"github-actions-
main","issuer":"https://token.actions.githubusercontent.com","subject":"rep
o:aintnier/azure-hub-spoke-terraform:ref:refs/heads/main","audiences":
["api://AzureADTokenExchange"]}'

az ad app federated-credential create \
  --id <APP_ID> \
  --parameters '{"name":"github-actions-
pr","issuer":"https://token.actions.githubusercontent.com","subject":"repo:
aintnier/azure-hub-spoke-terraform:pull_request","audiences":
["api://AzureADTokenExchange"]}'
```

```
az ad app federated-credential create \
  --id <APP_ID> \
  --parameters '{"name":"github-actions-environment-production","issuer":"https://token.actions.githubusercontent.com","subject":"repo:aintnier/azure-hub-spoke-terraform:environment:production","audiences":["api://AzureADTokenExchange"]}'
```





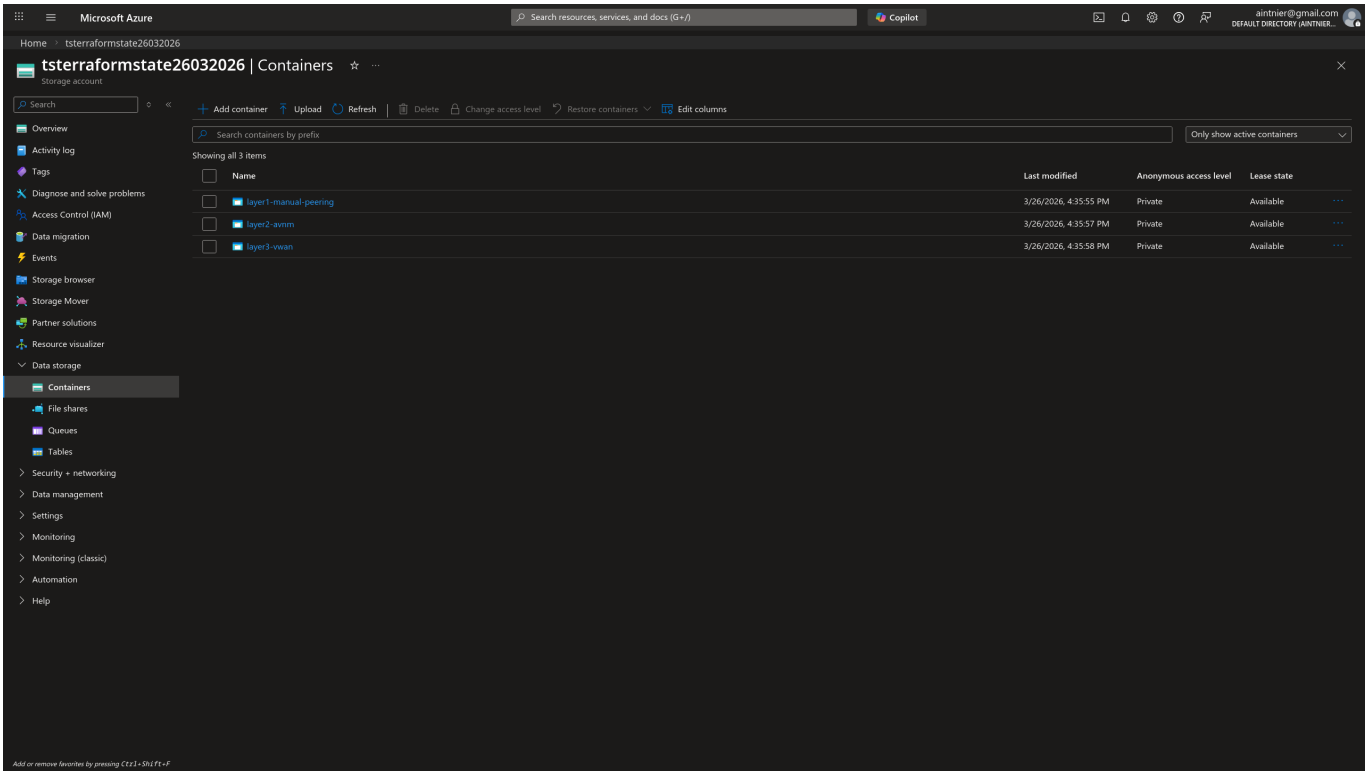
Remote Terraform State

Terraform state is isolated per layer in a dedicated Storage Account (`tsterraformstate26032026`):

Container	Layer
`layer1-manual-peering`	L1 - Manual VNet Peering
`layer2-avnm`	L2 - Azure Virtual Network Manager
`layer3-vwan`	L3 - Azure Virtual WAN

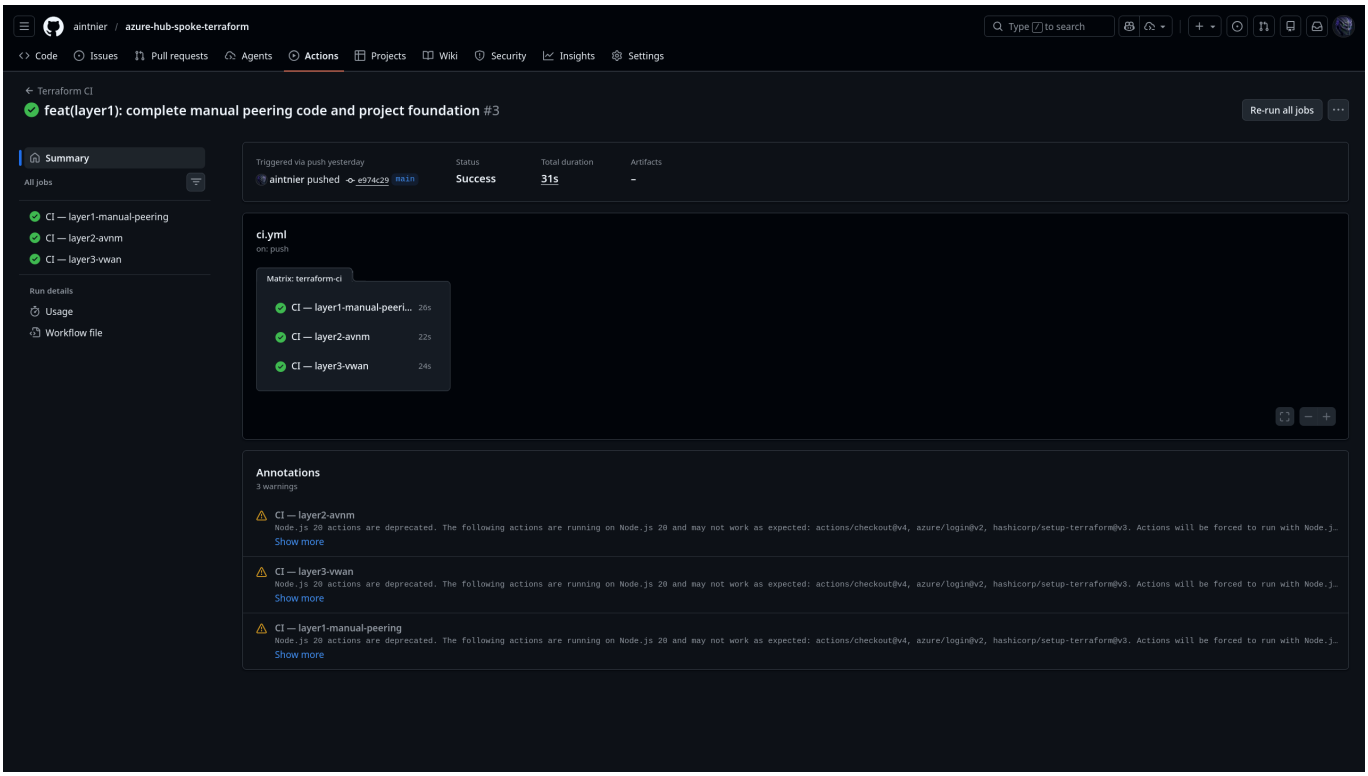
▶ Azure CLI setup commands

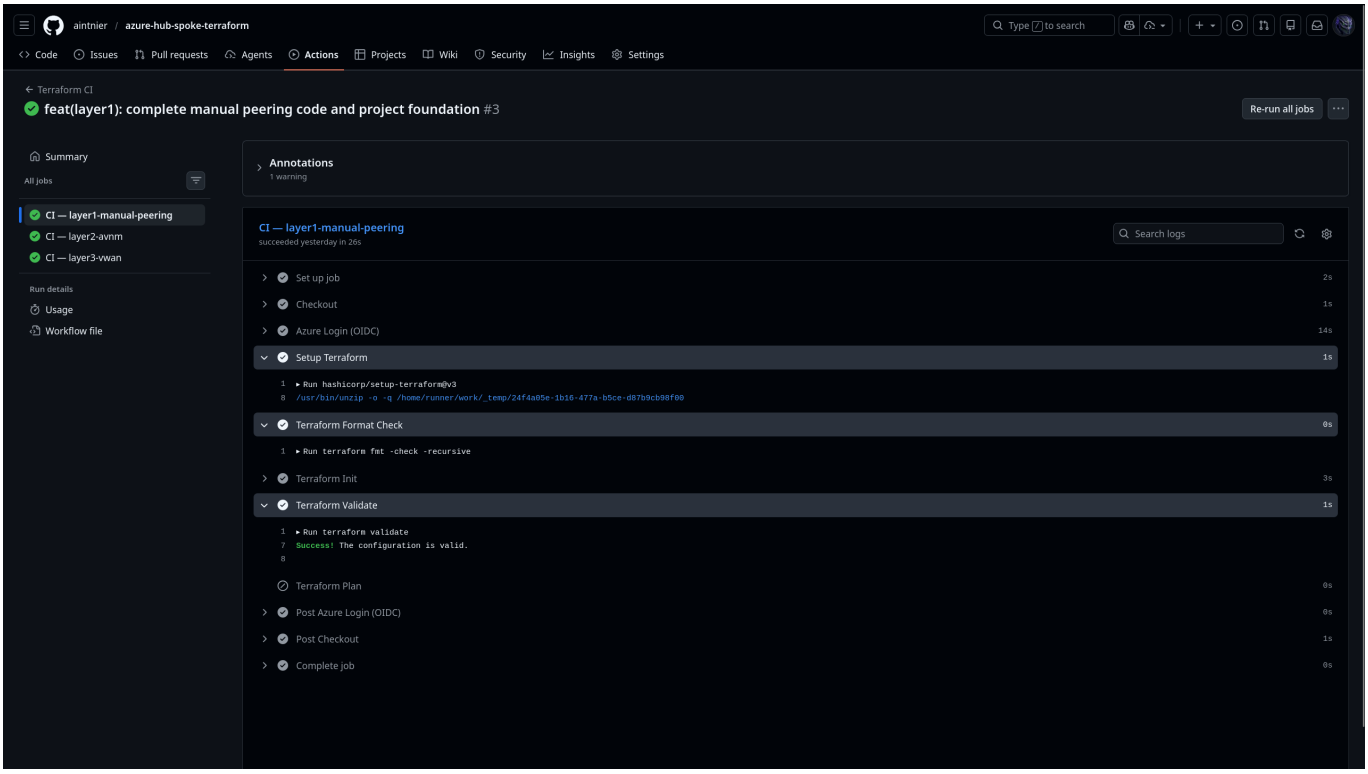
```
az group create --name rg-terraform-state --location westeurope

az storage account create \
  --name tsterraformstate26032026 \
  --resource-group rg-terraform-state \
  --location westeurope \
  --sku Standard_LRS

az storage container create --name layer1-manual-peering --account-name
tsterraformstate26032026
az storage container create --name layer2-avnm --account-name
tsterraformstate26032026
az storage container create --name layer3-vwan --account-name
tsterraformstate26032026
```



Continuous Integration (CI)

The GitHub Actions matrix workflow (`ci.yml`) runs `terraform fmt`, `terraform init`, and `terraform validate` across all three layers **in parallel** on every push and pull request - without contacting Azure state.



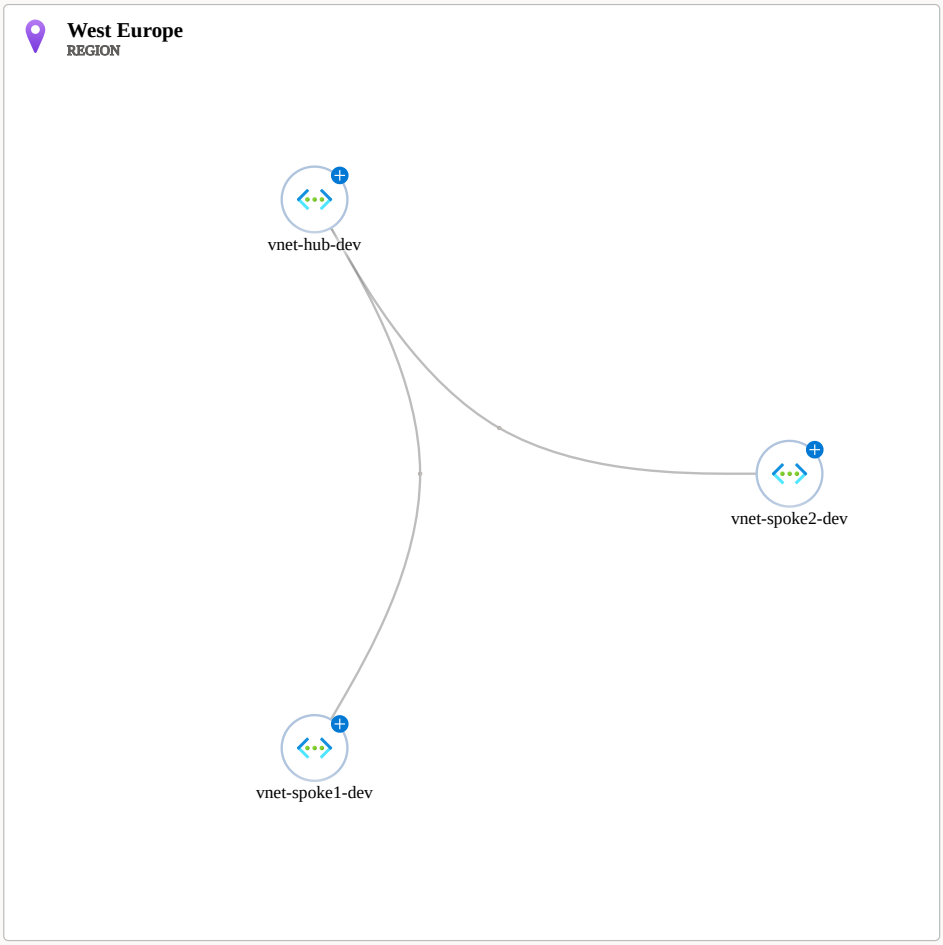


Layer Details

Layer 1 - Manual VNet Peering

Layer 1 Manual VNet Peering | Docs RUNBOOK

Explicit hub-to-spoke peerings with UDR-based traffic steering. Every routing decision is hand-declared in Terraform - zero abstraction, maximum debuggability. Traffic inspection is enforced by routing `0.0.0.0/0` on spoke subnets to the Firewall private IP (`10.0.0.4`).



Key resources provisioned:

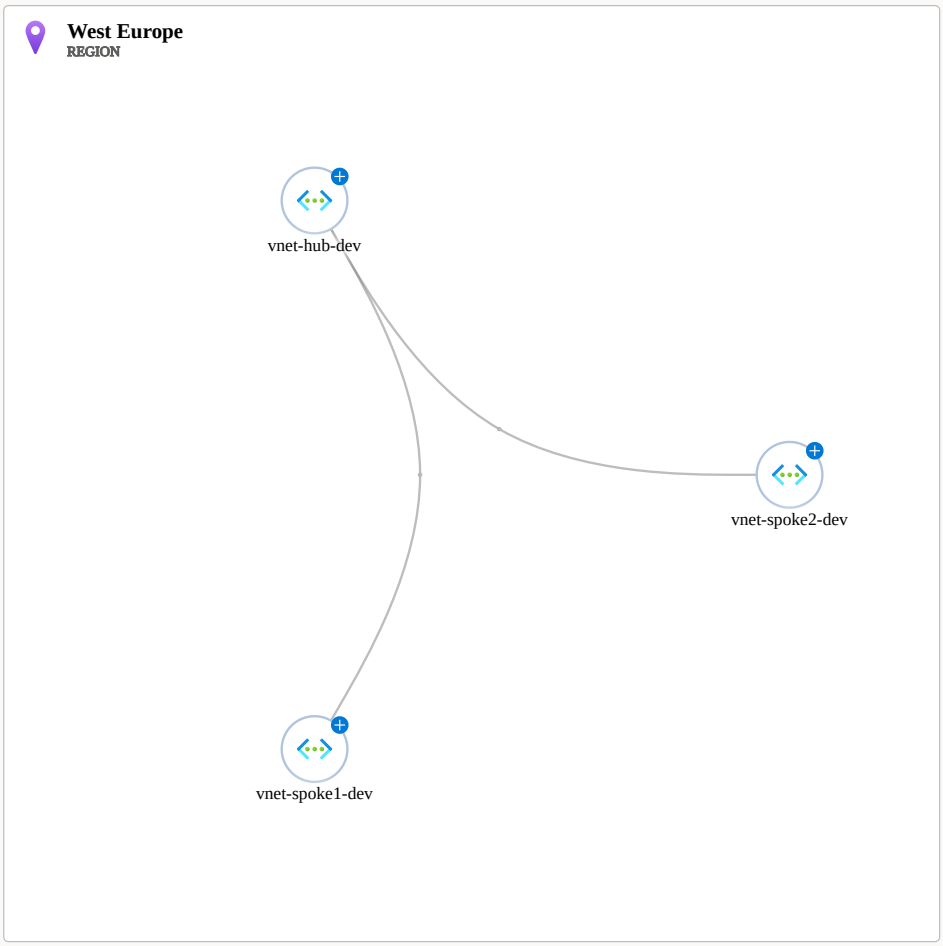
- Hub VNet + `AzureFirewallSubnet` + Azure Firewall (Standard)
- Spoke 1 VNet + Spoke 2 VNet
- Manual hub-to-spoke VNet Peerings (x2)
- Route Tables (UDR) on each spoke subnet - `0.0.0.0/0` to `10.0.0.4`
- Azure Bastion (Hub) + Test VMs (Ubuntu 22.04, Standard_D2s_v3)
- Log Analytics Workspace + Firewall Diagnostic Settings

For deployment notes, validation tests, and lessons learned see: `layer1-manual-peering/RUNBOOK.md`

Layer 2 - Azure Virtual Network Manager (AVNM)

Layer 2 | AVNM Dynamic Groups | Docs | RUNBOOK

AVNM replaces explicit peering resources with **tag-based dynamic group membership**. Spokes join the hub-spoke topology automatically when tagged `avnm-group = 'hub-spoke-layer2'` - no peering code required per new spoke. UDRs are still maintained for traffic steering to the Firewall.



Key architectural detail - AVNM publish drift:

AVNM separates *configuration definition* from *configuration publish*. A successful `terraform apply` does not guarantee Azure has activated the new topology. The `triggers` block below prevents silent drift:

```
triggers = {
  configuration_ids = join(",",
[azurerm_network_manager_connectivity_configuration.hub_and_spoke.id])
}
```

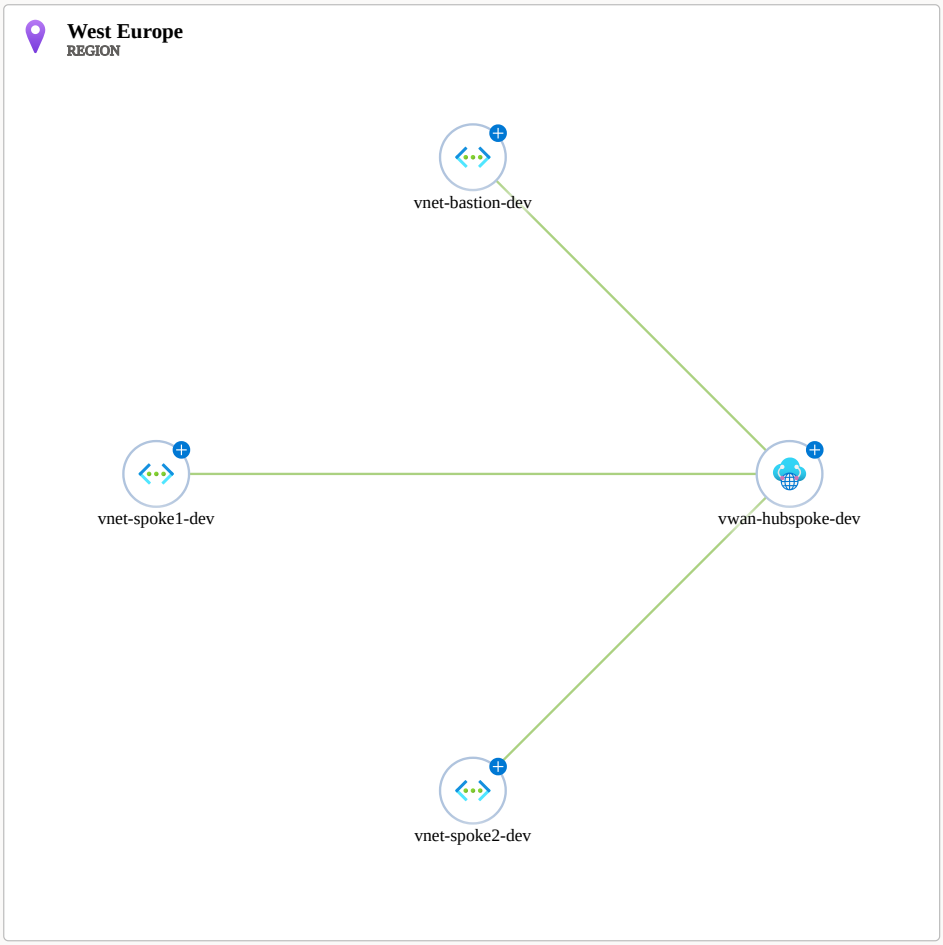
Without this block, Terraform can report green while spokes are still running a stale AVNM topology.

For full AVNM gotchas, RBAC issues, and deployment notes see: `layer2-avnm/RUNBOOK.md`

Layer 3 - Azure Virtual WAN (vWAN)

Layer 3 Azure Virtual WAN | Docs RUNBOOK

Azure Firewall is deployed as an integrated component of a **Secured Virtual Hub**. **Routing Intent** automatically programs the hub router to intercept all private (RFC 1918) and internet traffic - eliminating the need for any custom UDR in Terraform.



Key architectural decision - Bastion isolation:

When Routing Intent is enabled, Azure propagates `0.0.0.0/0` to all connected VNets. Because Bastion requires a symmetric inbound management path from the public internet, injecting a default route into the Bastion data plane breaks connectivity. The fix:

```
# azurerm_virtual_hub_connection - Bastion VNet
internet_security_enabled = false
```

This quarantines the Bastion VNet from Routing Intent's default-route injection while preserving full Zero-Trust enforcement on workload spokes.

> **Cost note:** This layer incurs significant baseline hourly charges for the Virtual Hub and managed Azure Firewall. Always run `terraform destroy` after testing.

For full vWAN behavior, Bastion SKU upgrade notes, and routing lessons see: `layer3-vwan/RUNBOOK.md`

Quick Start

Prerequisites

Requirement	Version
Azure Subscription	Contributor access
Terraform	>= 1.5
Azure CLI	>= 2.50
SSH key pair	`~/.ssh/id_rsa.pub`

Deploy a Layer

```
cd layer1-manual-peering   # or layer2-avnm / layer3-vwan
terraform init
terraform plan -out=tfplan
terraform apply tfplan

# Tear down when validation is done
terraform destroy -auto-approve
```

CI/CD Workflows

Apply and destroy workflows are **manual by design** to prevent accidental infrastructure changes triggered by automated CI runs.

Workflow	Trigger	Purpose
`ci.yml`	Push / PR to `main`	`fmt` + `init` + `validate` - all layers in parallel

Workflow	Trigger	Purpose
`deploy-layer1.yml`	Manual	`terraform apply` - Layer 1
`deploy-layer2.yml`	Manual	`terraform apply` - Layer 2
`deploy-layer3.yml`	Manual	`terraform apply` - Layer 3
`destroy-layer1.yml`	Manual	`terraform destroy` - Layer 1
`destroy-layer2.yml`	Manual	`terraform destroy` - Layer 2
`destroy-layer3.yml`	Manual	`terraform destroy` - Layer 3

Authentication uses **OIDC Federated Identity** on all workflows. No static client secret is stored anywhere in the repository.

Known Limitations

- **VM SKU availability:** `Standard_B1s` is not always available in West Europe. The Terraform declarations use `Standard_D2s_v3` as a fallback - adjust per your region's capacity.
- **AVNM propagation lag:** AVNM connectivity configuration deployments can take several minutes to propagate across the Azure fabric. A green `terraform apply` does not guarantee the topology is immediately active.
- **Bastion SKU on vWAN:** Layer 3 requires Azure Bastion **Standard SKU** to support IP-based connections across the segmented topology. Basic SKU is not sufficient.
- **Routing Intent scope:** Enabling Routing Intent on a vWAN Secured Hub applies a blanket `0.0.0.0/0` default route to all connected VNets. Management subnets such as Bastion must be explicitly opted out via `internet_security_enabled = false` to avoid asymmetric routing failures.

Repository Structure

```
.
├── .github/
│   └── workflows/            # 7 CI/CD pipelines (ci, deploy x3, destroy
x3)
├── docs/
│   └── imgs/                 # Architecture diagrams and validation
screenshots
│       ├── setup/
│       ├── layer1-manual-peering/
│       ├── layer2-avnm/
│       └── layer3-vwan/
├── layer1-manual-peering/    # VNet Peering + Azure Firewall + UDR + VMs
│   └── RUNBOOK.md
├── layer2-avnm/              # AVNM dynamic groups + Azure Firewall + VMs
│   └── RUNBOOK.md
├── layer3-vwan/              # Virtual WAN + Secured Hub + VMs
```

```
│   └── RUNBOOK.md
└── README.md
```

Each layer directory contains independent Terraform state, provider configuration, and a `RUNBOOK.md` with deployment session notes, validation tests, control-plane telemetry, and engineering lessons.

Tech Stack

Component	Technology
IaC	Terraform >= 1.5, AzureRM provider ~> 3.x
CI/CD	GitHub Actions (matrix workflow)
Cloud	Microsoft Azure (West Europe)
Authentication	OIDC Federated Identity Credential
Network Security	Azure Firewall Standard
Test VMs	Ubuntu 22.04 LTS (Standard_D2s_v3)
Observability	Azure Network Watcher, Log Analytics, KQL

License

MIT